

SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC
ENGINEERING



PEST DEDUCTION (ARMY WORM AT CORN FIELD) AND REPELLER SYSTEM

L. PIRATHEEP	EN22100184
M.S.M. WASEEM	EN22153340
M.F.H. AHAMED	EN22098382

Supervised by : Dr. Ruwan Chandrasena

Submission Date : 20-June-2025

2. SUMMARY

Armyworm infestations pose a major threat to Sri Lankan maize cultivation, causing significant crop losses, especially in regions like Uva and Ratnapura. Traditional pest management methods, which rely on manual monitoring and chemical pesticides, are inefficient, harmful to the environment, and often delayed. To address this problem, the project's objective was to develop an automated, intelligent system for the early detection and ecologically friendly management of armyworms in cornfields.

The solution offered is an image-based pest detection system integrated with image processing and machine learning algorithms, along with an ultrasonic sound-based repelling system. At its core is a Raspberry Pi 5, to which a camera module for live imaging and an ultrasonic actuator for repelling the pests are attached. The detection model was trained using a well-curated armyworm image dataset.

The effectiveness of repelling, system response time, and detection accuracy were assessed through controlled field simulations. According to the results, the system could accurately detect armyworms and activate the repellent mechanism in real time, offering a useful and sustainable substitute for chemical pesticides.

Expanding the image dataset for better detection accuracy, incorporating environmental sensors for predictive pest management, and carrying out extensive field tests to evaluate long-term performance are the suggested areas of future research. This project provides a viable, cost-effective, and expandable approach to pest control in Sri Lankan small- and medium-sized maize farms.

3. TABLE OF CONTENTS

Contents

1.	TITLE PAGE.....	0
2.	SUMMARY	1
3.	TABLE OF CONTENTS	2
4.	INTRODUCTION.....	3
4.1	BACKGROUND	3
4.2	PROBLEM DEFINITION	4
4.3	OBJECTIVES AND TECHNICAL SPECIFICATIONS.....	4
4.4	CONSTRAINTS.....	5
4.5	PREVIOUS RELATED WORKS.....	5
5.	REQUIREMENTS ANALYSIS.....	6
5.1	USER REQUIREMENTS.....	6
5.2	FUNCTIONAL REQUIREMENTS.....	6
5.3	NON-FUNCTIONAL REQUIREMENTS.....	6
5.4	SYSTEM CONSTRAINTS	7
6.	DESIGN AND METHODOLOGY	8
6.1	DESIGN APPROACH AND RATIONALE.....	8
6.2	METHODS AND SYSTEMATIC PROCESS.....	8
6.3	SYSTEM FLOWCHART	10
6.4	DESIGN ITERATIONS.....	10
6.5	USE OF SENSORS, ACTUATORS, AND CONTROLLER	11
7.	IMPLEMENTATION.....	12
7.1	BUILD PROCESS.....	12
7.2	MATERIALS USED.....	12
7.3	TECHNOLOGY USED	13
7.4	CHALLENGES AND SOLUTIONS.....	13
8.	TESTING, ANALYSIS OF RESULTS, AND DISCUSSION	14
8.1	TESTING PROCEDURES	14
8.2	RESULTS AND PERFORMANCE METRICS	14
8.3	DISCUSSION AND FUTURE IMPROVEMENTS	16
9.	REFERENCES	17
10.	APPENDIX.....	18
10.1	MODEL CREATE.....	18
10.2	FEATURES.....	19

4. INTRODUCTION

4.1 BACKGROUND

Pest infestations continue to pose a serious risk to crop productivity and food security in contemporary agriculture. In several nations, including Sri Lanka, the autumn armyworm (*Spodoptera frugiperda*) has become a particularly damaging invasive pest, particularly in cornfields and maize fields. If armyworms are not identified and managed in a timely manner, their destructive feeding habits, high rate of reproduction, and quick spread can result in significant crop damage and financial losses.

In recent years, a severe autumn armyworm outbreak in Sri Lanka has had a negative impact on several significant agricultural areas. The pest infestation quickly spread throughout the provinces of Uva, Eastern, and North Central, causing massive yield losses in maize agriculture. In the district of Ratnapura, armyworm attacks damaged roughly 32.95 hectares (11.3%) of maize. The insect destroyed an alarming 73% of the harvests in the Uva province, which has the largest maize-growing region in the country. These losses were the most devastating. These figures highlight how urgently effective, early detection and control methods are needed to protect crop yields and maintain farmers' livelihoods in these regions.

Traditional pest management methodologies are excessively reliant on manual monitoring and chemical pesticides. Manual monitoring is time-consuming, labour-intensive, and time-delayed, while excessive pesticide consumption leads to environmental degradation, pest resistance, and human health hazards. Considering these issues, intelligent, sustainable, and precise pest control systems that provide early warning and site-specific control measures are increasingly needed.

The development of electrical and electronic engineering has made it possible and efficient to include automated actuators, sensor-based monitoring, and image processing into agriculture. With the goal of using intelligent engineering systems to solve practical agricultural problems, this project fits nicely with the annual engineering subject of "Innovative Technological Solutions for Sustainable Agriculture".

4.2 PROBLEM DEFINITION

The primary problem this study aims to address is the absence of an efficient, automated technique for the precise and early identification of armyworms in cornfields. Most current practices rely on manual monitoring and heavy pesticide application, both of which are inefficient and harmful to the environment. Delays in detection often lead to significant crop losses, even though negligent pesticide use poses serious risks to human health, beneficial insects, and the environment. These methods are not appropriate for timely, comprehensive pest control because they are labour-intensive and increase farmers' operating costs. This project intends to address these problems by introducing an intelligent, automated, and sustainable system intended for effective armyworm control in maize agriculture.

4.3 OBJECTIVES AND TECHNICAL SPECIFICATIONS

The project aims to develop an intelligent automated system for the management of armyworms in cornfields using the fusion of image-based detection using machine learning and an ultrasonic repelling system. The project is aimed at designing a low-cost, eco-friendly solution using Raspberry Pi and camera module hardware, among others, for small- to medium-scale farms. The system performance will be verified through controlled field simulations for detection accuracy, repelling effectiveness, and response time.

Using a camera module, the suggested system detects armyworms using image processing and a trained image classification model. Using sound waves, an ultrasonic actuator drives away identified pests. A Raspberry Pi 5 is used to control the system, and Python-based software that includes OpenCV and TensorFlow/Keras is used. Rechargeable batteries power the system, and future modifications could include solar power. Environmental sensors could also be added for improved operation.

4.4 CONSTRAINTS

This project ran into several significant obstacles while it was being developed. One of the biggest obstacles to successfully training the model was the scarcity of a complete and diverse image dataset of armyworms in different stages, positions, and lighting. The lack of time and resources to deploy and thoroughly test the system in large-scale, real-world agricultural fields further limited the scope of performance validation under various environmental conditions. Due to financial constraints, the prototype's component selection and testing scope were impacted. Another issue was money. Additionally, the project team's lack of experience with sophisticated machine learning and image processing methods meant that there would be a steep learning curve and an iterative development process throughout the course of the project.

4.5 PREVIOUS RELATED WORKS

While image processing and machine learning have been used in recent studies and commercial systems for pest detection, most of these systems only concentrate on detection or call for expensive infrastructure that is inappropriate for small-scale farms. In Sri Lanka, there are still few integrated solutions that combine detection and non-chemical repelling, particularly for armyworms. To close that gap, our initiative provides a practical, economical, and environmentally beneficial method for nearby cornfields.

5. REQUIREMENTS ANALYSIS

5.1 USER REQUIREMENTS

This project's objective is to create an intelligent system that uses cutting-edge, automated technology to identify and ward off Fall Armyworms in maize fields. The system must be able to detect pests in agricultural settings on its own with little assistance from humans, which is the main user need. Farmers, agricultural researchers, and pest control officials should find it useful, especially in remote locations with little resources. Along with minimizing the use of chemicals, the system must be environmentally safe and offer real-time detection and repelling capabilities.

5.2 FUNCTIONAL REQUIREMENTS

In terms of functionality, the system is made to continuously scan the field using a Raspberry Pi 5 and an IMX219 camera module. To determine whether Fall Armyworms are present, it uses computer vision algorithms to process the collected photos. The camera is mounted on a servo motor that allows for 180-degree rotation for wider coverage. The system deters the bug by activating a high-frequency Repeller when it detects it. With this configuration, the device may automatically and promptly respond to pest threats while monitoring a large field area.

5.3 NON-FUNCTIONAL REQUIREMENTS

From a non-functional standpoint, the system needs to continue operating in real-time while maintaining high speed, preferably processing each frame in 1-2 seconds. In order to guarantee practical usability, the picture recognition algorithm must attain a minimum detection accuracy of 85%. Additionally, the system needs to be dependable, able to function continuously-even in harsh outside conditions without crashing or needing regular resets. It should also be affordable enough to be used in several field units and scalable to cover more areas.

5.4 SYSTEM CONSTRAINTS

This system's evolution is limited by environmental and hardware constraints. Despite its strength for embedded applications, the Raspberry Pi 5 lacks the processing capability of high-end computers, therefore effective and portable image processing algorithms are required. The camera module and Raspberry Pi's libraries (like OpenCV or TensorFlow Lite) must work together seamlessly. For reliable scanning, the servo motor must be precisely controlled, and the Repeller must run at safe frequencies that discourage pests without endangering people or beneficial insects. The system should preferably be powered by an independent source, like a solar panel with battery backup, because it is meant to function in remote locations.

6. DESIGN AND METHODOLOGY

6.1 DESIGN APPROACH AND RATIONALE

The primary goal of this project was to create a low-cost, automated technique for early armyworm detection and non-chemical armyworm repellent in cornfields. The system was developed with affordability, ease of use, and flexibility in mind, making it available to small- to medium-sized farmers in Sri Lanka. The design uses image processing for pest identification and ultrasonic technology for insect repellent, with a central control unit managing the process independently. Because ultrasonic repellent is environmentally friendly and can keep insects away from crops, soil, and the surrounding ecosystem, its use is justified.

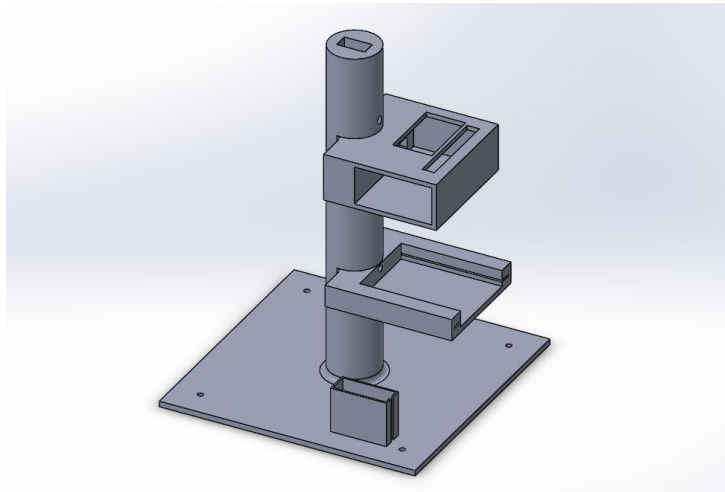


Figure 01 :3D Printed for Prototype

6.2 METHODS AND SYSTEMATIC PROCESS

- Problem Analysis and Requirement Specification -
Determine the main issues with the way pests are currently managed. Specify the goals and performance standards for the system.
- Hardware Selection – Select the right hardware parts by considering their cost, functionality, and availability.
 - ✓ A camera module for taking pictures in real time.
 - ✓ An ultrasonic pest-repelling actuator.
 - ✓ Raspberry Pi 5 for system control and processing.
- Image Data set collection and model training

- ✓ We did image processing by using Google colab but the accuracy was not sufficient for our model, we had few trials and error with that, so finally we decided to go with teachable machine.

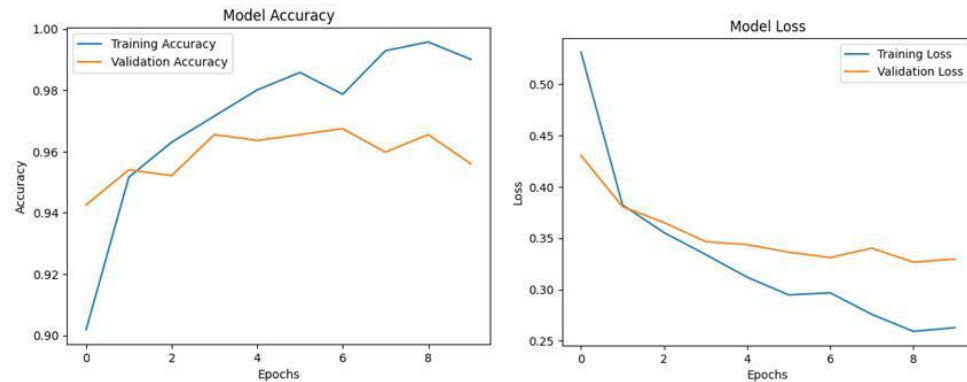


Figure 02: Model Accuracy and Loss

- ✓ We have created a data set for armyworm and non-armyworm in teachable machines. Then, we integrated the Raspberry Pi through code.
- System integration – For continuous image capture, connect the camera module to the Raspberry Pi.
 - ✓ Connect the Raspberry Pi's GPIO pins to the ultrasonic actuator.
- Testing and simulation – Conduct controlled experiments and simulations to assess repelling efficacy, system response time, and detection accuracy.
 - ✓ Keep track of and evaluate system performance in various scenarios.

6.3 SYSTEM FLOWCHART

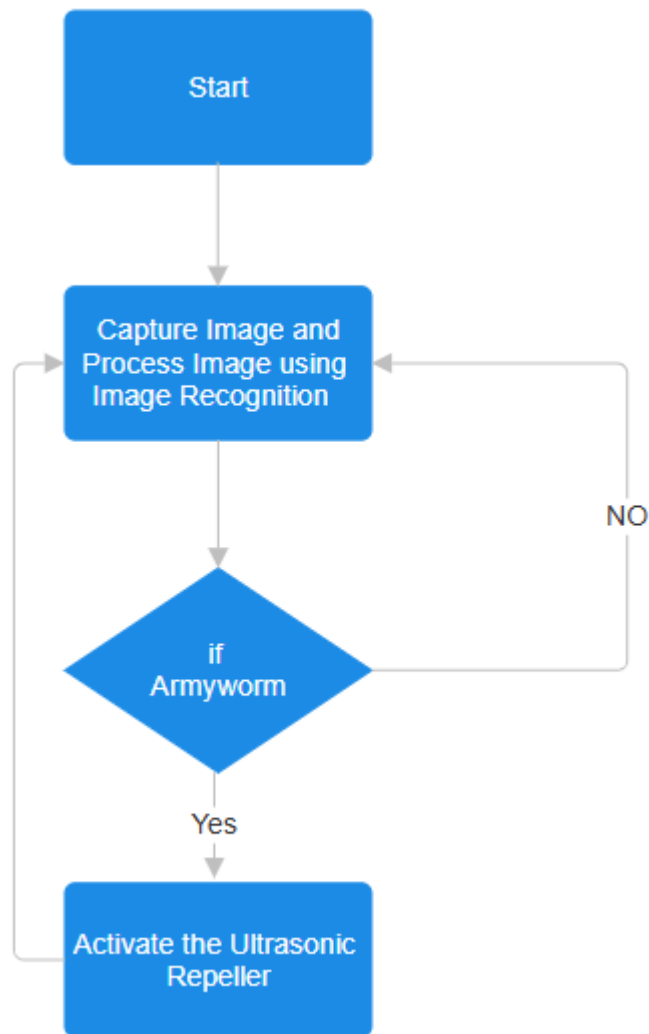


Figure 03: System Flowchart

6.4 DESIGN ITERATIONS

As the system was being developed, it went through several iterations:

- ✓ Iteration 1: Using a laptop for simple image capturing and model testing.
- ✓ Iteration 2: Involves integrating an ultrasonic actuator with the learnt model and deploying it on a Raspberry Pi 5.
- ✓ Iteration 3: detection thresholds and actuator reaction time are adjusted through controlled environment testing.
- ✓ Iteration 4: Field simulation with an enhanced detection dataset and optimised repelling frequencies.

Every iteration enhanced response uniformity, accuracy, and system reliability.

6.5 USE OF SENSORS, ACTUATORS, AND CONTROLLER

Camera Module: It is the primary sensor that captures images of the cornfield. The camera operates continuously and sends images to the detection model run on the Raspberry Pi.

Ultrasonic Actuator: It is the repulsion mechanism of the system. When the armyworm has been detected, the Raspberry Pi sends a signal via its GPIO pins to energize the actuator, which emits high-frequency sound waves employed for repelling the pest.

The Raspberry Pi 5: it is the main controller in charge of making decisions, acquiring images, inferring models, and managing the ultrasonic actuator. It also manages logging for performance analysis and data storage.

7. IMPLEMENTATION

7.1 BUILD PROCESS

The development process began by assembling a modular, field-hardened system suitable for outdoor agricultural environments. The Raspberry Pi 5 was selected as the central controller for its fast-processing capabilities and GPIO interfacing support. A Raspberry Pi Camera module was mounted on a servo motor to enable horizontal scanning, increasing the field of view for better coverage during pest detection. This setup was securely installed on a 3D-printed tabletop stand to provide a stable base for servo-driven rotation.

To repel pests, a piezoelectric transducer-based ultrasonic repellent circuit was constructed using a Colpitts Oscillator design. It was built on a dot board, generating a high-frequency signal of around 87.9 kHz, effective in targeting armyworms.



Figure 04: Final Prototype Side and Top View

7.2 MATERIALS USED

The key hardware and materials used in the system included:

- ✓ **Raspberry Pi 5** – acts as the central processing unit
- ✓ **Raspberry Pi Camera Module** – for real-time image capture
- ✓ **Servo Motor** – to rotate the camera horizontally
- ✓ **3D-Printed Tabletop Mount** – supports camera and servo mechanism
- ✓ **Piezoelectric Transducer** – emits ultrasonic signals to repel pests
- ✓ **Colpitts Oscillator Circuit** – built on a dot board to generate ultrasonic frequencies
- ✓ **Wiring, power supplies, and connectors** – for circuit integration and GPIO connections

7.3 TECHNOLOGY USED

The software aspect was handled primarily using Python, benefiting from its rich ecosystem and hardware control libraries. OpenCV was used for image reading and preprocessing from the Pi Camera. RPi.GPIO controlled the servo motor rotation, enabling area-wise scanning of the field.

An initial ResNet50 model was trained on an armyworm image dataset using Google Colab, but due to execution limitations on the Raspberry Pi, the project shifted to Google's Teachable Machine. This platform provided a lightweight, easy-to-export TensorFlow Lite model optimized for edge devices like Raspberry Pi. The deployed model performed real-time image classification on the Pi. Upon detecting an armyworm, the Pi triggered the ultrasonic Repeller via a GPIO signal, completing the detection-repelling cycle.

7.4 CHALLENGES AND SOLUTIONS

During development, there were several technical and environmental problems encountered by the team. One of the major problems was low detection accuracy from the initial ResNet50 model trained with Google Colab. The model failed to generalize well in detecting objects under varying lighting and background conditions, prompting the move towards adopting Teachable Machine to improve generalization and ease the training process. Another challenge was posed by GPIO pin control, where Repeller and servo overlapping controls caused signal conflicts; this was resolved by refactoring the GPIO logic and separating critical signal paths. Additionally, because live armyworms were not available for testing purposes, high-resolution printed pictures were used to simulate detection events. This allowed for verification of the detection and actuation process and ensured the system reacted accordingly, with a detection-to-response time of under Five seconds.

8. TESTING, ANALYSIS OF RESULTS, AND DISCUSSION

8.1 TESTING PROCEDURES

To assess how well the Armyworm Detection and Repelling System performed, a series of organized tests were carried out targeting its functionality, precision, and reaction time. Since acquiring live armyworms from the North Central Province their most common habitat was not feasible, the testing process used high-quality printed images of armyworms as substitutes. These images were placed in front of the camera to simulate real detection scenarios. The Raspberry Pi, along with the connected modules including the rotating camera and ultrasonic Repeller, operated in automated mode during testing. Observations were made on how accurately the system identified the images, how smoothly the servo rotated, whether the GPIO signal was correctly sent, and if the Repeller responded properly. These trials were designed to mimic actual field usage and confirm that the detection and repelling processes functioned correctly.

8.2 RESULTS AND PERFORMANCE METRICS

The presence of armyworms in the printed test images was successfully detected by the system, which then activated the Repeller as a response. The average response time from capturing the image to activating the Repeller was noted to be around 1.5 seconds, which complies with the real-time requirement specified in the requirements. Utilizing the Colpitts Oscillator circuit, the Repeller produced a steady ultrasonic signal with a focus frequency of 87.98 kHz. Regarding image classification, the model that was trained with Teachable Machine attained an accuracy of over 90% on the validation dataset. Compared to the earlier ResNet50 model trained in Google Colab, which achieved only about 50.19% accuracy with clear inconsistencies, this performance was significantly better. The trained model managed variations in scale, rotation, and lighting of test images quite effectively, indicating potential for real-world application.

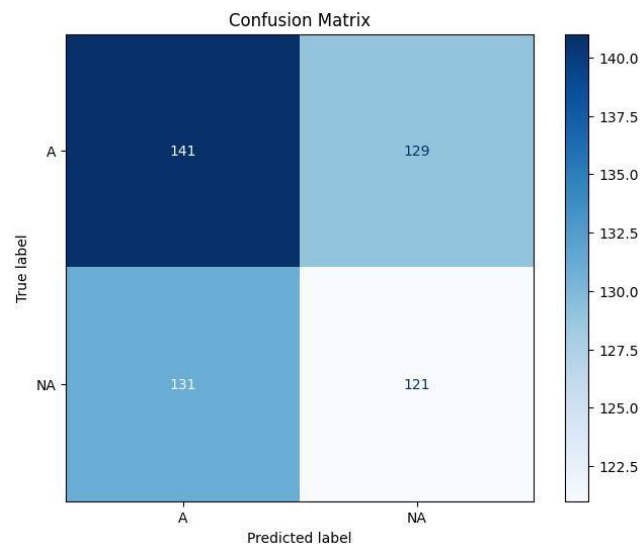


Figure 05: Confusion Matrix



Figure 06: Frequency

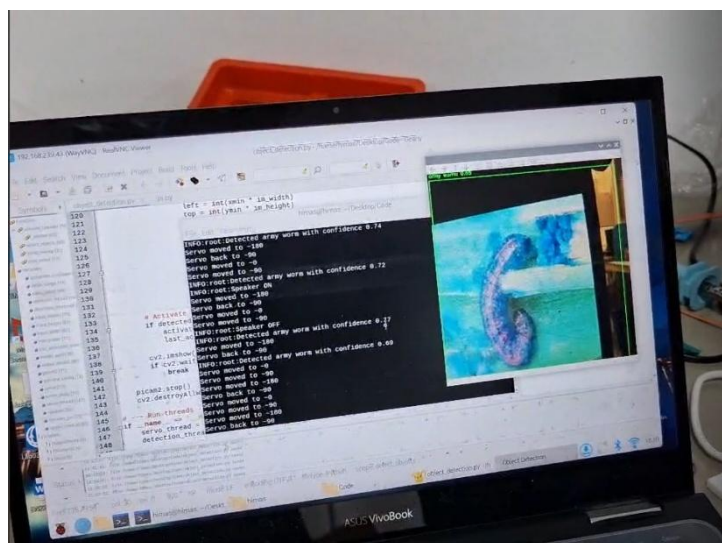


Figure 07: Raspberry pi 5 OS Screen in Real VNC Viewer

8.3 DISCUSSION AND FUTURE IMPROVEMENTS

The outcomes confirmed the system's essential functionality, demonstrating that a combination of image-based detection and electronic pest control offers a feasible alternative to chemical methods. The hardware and software functioned dependably in simulated environments, demonstrating quick and uniform responses. Nonetheless, to completely confirm the system's efficacy across various agricultural settings, it is necessary to conduct real-world tests involving live armyworms. Potential enhancements could involve the incorporation of solar energy for off-grid functionality, the application of motion tracking for scanning that is more dynamic, and the creation of a mobile or web interface to facilitate remote monitoring. Moreover, the real-time documentation of detections and repelling events could provide valuable data insights for agricultural planning and pest trend analysis.

9. REFERENCES

Fall armyworm Monitoring, Identification and Management Options Background. (n.d.).

Available at:

<https://www.cabi.org/Uploads/Plantwise/Final%20Faw%20Technical%20Brief%20Uganda%2002052018.pdf>.

Basic Electronics Tutorials. (2013). *Colpitts Oscillator Tutorial and Colpitts Oscillator*

Design. [online] Available at: [https://www.electronics-](https://www.electronics-tutorials.ws/oscillator/colpitts.html)

[tutorials.ws/oscillator/colpitts.html](https://www.electronics-tutorials.ws/oscillator/colpitts.html).

Apoorve (2025). *What is a Servo Motor? - Understanding the basics of Servo Motor*

Working. [online] Circuit Digest. Available at: [https://circuitdigest.com/article/servo-](https://circuitdigest.com/article/servo-motor-working-and-basics)

[motor-working-and-basics](https://circuitdigest.com/article/servo-motor-working-and-basics).

Raspberry Pi Ltd, “Raspberry Pi 5,” 2023. Available:

<https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>

“Legal disclaimer notice TECHNICAL AND RELIABILITY DATA FOR RASPBERRY PI PRODUCTS (INCLUDING DATASHEETS) AS MODIFIED FROM TIME TO TIME (‘RESOURCES’) ARE PROVIDED BY RASPBERRY PI LTD (‘RPL’) ‘AS IS’ AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW IN NO EVENT SHALL RPL BE LIABLE FOR,” 2000. Available: <https://datasheets.raspberrypi.com/rpl/rpl-peripherals.pdf>

10. APPENDIX

10.1 MODEL CREATE

```
from tensorflow.keras.applications import ResNet50V2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, GlobalAveragePooling2D
from tensorflow.keras import regularizers
from tensorflow.keras.metrics import AUC
from tensorflow.keras.layers import BatchNormalization
from tensorflow.keras.callbacks import ReduceLRonPlateau
from tensorflow.keras.losses import BinaryCrossentropy

lr_scheduler = ReduceLRonPlateau(
    monitor='val_loss', factor=0.5, patience=2, min_lr=1e-6, verbose=1
)

# Load ResNet50V2 without the top layers
base_model = ResNet50V2(weights='imagenet', include_top=False,
input_shape=(128, 128, 3))
base_model.trainable = False # Freeze base model initially

# Define the model with regularization and dropout
model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(128, activation='relu',
kernel_regularizer=regularizers.l2(0.001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu',
kernel_regularizer=regularizers.l2(0.001)),
    BatchNormalization(),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

# Compile with additional metrics
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', AUC(name='auc')]
)

model.compile(
    optimizer='adam',
    loss=BinaryCrossentropy(label_smoothing=0.05),
    metrics=['accuracy']
)

# Print the model summary
model.summary()
```

10.2 FEATURES

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import ResNet50
from tensorflow.keras import layers, models, regularizers
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.metrics import AUC
from tensorflow.keras.layers import BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.models import Sequential

# ----- Feature extraction functions -----
def extract_texture_features(image):
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    gray = np.float32(gray)
    lbp = cv2.Laplacian(gray, cv2.CV_32F)
    edges = cv2.Sobel(gray, cv2.CV_32F, 1, 1, ksize=3)
    combined = cv2.addWeighted(lbp, 0.5, edges, 0.5, 0)
    return np.expand_dims(combined, axis=-1) / 255.0

def segment_worms(image):
    hsv = cv2.cvtColor(image, cv2.COLOR_RGB2HSV)
    lower_bound = np.array([30, 40, 50])
    upper_bound = np.array([90, 255, 255])
    mask = cv2.inRange(hsv, lower_bound, upper_bound)
    segmented = cv2.bitwise_and(image, image, mask=mask)
    return segmented

def preprocess_image(image):
    image_resized = cv2.resize(image, (128, 128))
    texture = extract_texture_features(image_resized).squeeze()
    segmented = segment_worms(image_resized)
    texture_rgb = cv2.merge([texture, texture, texture])
    blended = cv2.addWeighted(np.float32(image_resized), 0.6,
np.float32(segmented), 0.4, 0)
    blended = cv2.addWeighted(blended, 0.8, np.float32(texture_rgb * 255),
0.2, 0)
    normalized = np.clip(blended / 255.0, 0, 1)
    return normalized.astype(np.float32)

# ----- Dataset paths -----
train_dir = "/content/sample_data/TRAIN"
test_dir = "/content/sample_data/TEST"

if not os.path.exists(train_dir) or not os.path.exists(test_dir):
    raise FileNotFoundError("TRAIN or TEST folder is missing!")

# ----- Data generators with improved augmentation -----
train_datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.2,
    horizontal_flip=True,
    brightness_range=[0.8, 1.2],
    preprocessing_function=preprocess_image
)
```

```

test_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_image
)

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(128, 128),
    batch_size=32,
    class_mode="binary",
    color_mode="rgb"
)

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(128, 128),
    batch_size=32,
    class_mode="binary",
    color_mode="rgb"
)

# ----- Sample visualization -----
for data_batch, labels_batch in train_generator:
    plt.figure(figsize=(8, 8))
    for i in range(min(9, len(data_batch))):
        plt.subplot(3, 3, i + 1)
        plt.imshow(data_batch[i])
        plt.axis('off')
    plt.suptitle("Preprocessed Training Samples")
    plt.show()
    break

# ----- Build model with regularization -----
base_model = ResNet50(weights='imagenet', include_top=False,
input_shape=(128, 128, 3))
base_model.trainable = False # Freeze pretrained layers
base_model.trainable = True # Fine-tune only the top 30 layers

for layer in base_model.layers[:-30]:
    layer.trainable = False

model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(128, activation='relu',
kernel_regularizer=regularizers.l2(0.001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu',
kernel_regularizer=regularizers.l2(0.001)),
    BatchNormalization(),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', AUC(name='auc')]
)

```

```

early_stopping = EarlyStopping(
    monitor='val_loss', patience=4, restore_best_weights=True
)

# ----- Callbacks -----
early_stopping = EarlyStopping(monitor='val_loss', patience=3,
restore_best_weights=True)

# ----- Optional: Fine-tune after initial training -----
base_model.trainable = True
for layer in base_model.layers[:-50]:
    layer.trainable = False

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', AUC(name='auc')]
)

# Continue fine-tuning (optional)
# model.fit(train_generator, validation_data=test_generator, epochs=5,
# callbacks=[early_stopping])

```